

ASIGNATURA:	PROGRAMACION JAVA	NOTA:	
NOMBRE:		FECHA:	26-08-2026
PROGRAMA:	TECNOLOGIA EN DESARROLLO DE SISTEMAS INFORMATICOS	DOCUMENTO:	
No.	COMPETENCIAS EVALUADAS	RESULTADOS DE APRENDIZAJE ESPERADOS	POND.
1	Desarrollo de aplicaciones web dinámicas utilizando Java, JSP.	El estudiante será capaz de diseñar e implementar una aplicación web completa para la administración de una inmobiliaria, con página de aterrizaje, autenticación mediante contraseñas cifradas, control de acceso por roles diferenciados (administrador, inmobiliaria, cliente y visitante), gestión de propiedades, citas y solicitudes, asegurando funcionalidad integral y un manejo seguro de las sesiones.	40
2	Aplicación de metodología en el desarrollo de software	El estudiante demostrará la capacidad de planificar y ejecutar un proyecto en tres sprints de 7 días cada uno, documentando planning, review y retrospective, gestionando un product backlog con historias de usuario priorizadas y utilizando herramientas como Git y un tablero de seguimiento, simulando los roles Scrum en un entorno iterativo.	30
3	Diseño de bases de datos y documentación en proyectos web	El estudiante diseñará un modelo de datos normalizado con relaciones uno a uno, uno a muchos y muchos a muchos, llaves primarias y foráneas y restricciones UNIQUE; lo integrará en una base de datos local y en línea mediante JDBC, y producirá la documentación completa (MER, modelo relacional, diccionario de datos y consultas), con pruebas unitarias y sustentación individual, asegurando un diseño responsivo y escalable.	30
RECOMENDACIONES			
✓ Apague su celular y/o guárdelo en su bolso. ✓ No se aceptan reclamos para respuestas en lápiz. ✓ No se permite el prestamos de lápices, borradores o calculadoras.		✓ No se permite la consulta de apuntes. ✓ El análisis hace parte del parcial, no se responden preguntas de procedimiento.	

Objetivo General: Diseñar e implementar una aplicación web dinámica para la administración de una inmobiliaria, utilizando Java EE, JSP (Java Server Pages) y JDBC sobre una base de datos relacional normalizada. El sistema debe contar con un módulo de autenticación que separe los privilegios por rol y con un modelo de datos que evidencie relaciones uno a uno, uno a muchos y muchos a muchos, además de restricciones de integridad. El desarrollo se realizará bajo el marco de trabajo Scrum, en tres sprints de siete días cada uno, para fomentar la planificación, la iteración y la entrega incremental. El despliegue del sitio y de la base de datos en línea otorga puntos adicionales.

Descripción del Proyecto: Construirá el sistema web de una inmobiliaria ficticia que gestione propiedades (casas, apartamentos, locales, oficinas y terrenos), usuarios con roles diferenciados y las operaciones asociadas: publicación del inmueble, búsqueda con filtros, agendamiento de visitas y trámite de solicitudes de compra o arriendo. Como mínimo, la aplicación debe contemplar los siguientes módulos:

- Página de aterrizaje (landing page):** página pública y responsiva con la presentación de la inmobiliaria, un buscador rápido de propiedades, las publicaciones destacadas y los accesos a registro e inicio de sesión.
- Módulo de autenticación:** registro de usuarios, inicio y cierre de sesión, y redirección automática al panel que corresponda según el rol asignado.
- Paneles (dashboards) diferenciados:** cada rol accede únicamente a las opciones y a la información que le competen.
- Gestión de propiedades:** listado con filtros, ficha de detalle con galería de imágenes y características, formulario de creación y edición, y baja lógica del inmueble.
- Gestión de visitas y solicitudes:** agendamiento de citas, radicación de documentos por parte del cliente y aprobación o rechazo por parte de la inmobiliaria.
- Reportes:** consultas consolidadas (propiedades disponibles por ciudad, citas por estado, solicitudes por inmobiliaria) construidas con sentencias SQL que involucren varias tablas.

Roles del Sistema: el sistema manejará como mínimo cuatro roles. Los permisos deben controlarse tanto en la interfaz como en el servidor:

- Visitante (no autenticado):** navega la landing page, consulta el catálogo público y el detalle de las propiedades. No accede a los paneles internos ni a los datos de contacto completos.
- Cliente:** busca y filtra propiedades, marca favoritos, solicita citas, radica los documentos de compra o arriendo, consulta el estado de sus trámites y actualiza su perfil.
- Inmobiliaria (agente):** publica, edita y da de baja sus propiedades, administra la galería de imágenes y las características del inmueble, atiende las solicitudes de cita, revisa y aprueba la documentación de los clientes y genera reportes de ventas y arriendos.

- **Administrador:** acceso total. Gestiona usuarios y asignación de roles, activa o inactiva cuentas, parametriza los catálogos del sistema (tipos de propiedad, ciudades, características) y consulta la auditoría de la aplicación.

Autenticación, Sesiones y Seguridad: el ingreso al sistema es la puerta que separa los roles, por lo que debe implementarse con los siguientes criterios de obligatorio cumplimiento:

- Las contraseñas se almacenan cifradas mediante una función de hash (BCrypt, PBKDF2 o SHA-256 con salt). Queda prohibido guardarlas en texto plano.
- La validación de credenciales se realiza contra la base de datos y la sesión HttpSession conserva el identificador del usuario y el o los roles que tenga asignados.
- Un filtro de servlet (Filter) debe proteger las rutas privadas: si un usuario no autenticado, o autenticado sin el rol requerido, intenta ingresar escribiendo la URL directamente, el sistema lo redirige a una página de acceso denegado.
- Los menús y botones se construyen dinámicamente según el rol, pero la validación en el servidor es obligatoria: ocultar una opción en la vista no se considera control de acceso.
- **Valor agregado (opcional):** bloqueo temporal de la cuenta tras varios intentos fallidos, recuperación de contraseña por correo y registro de la actividad en la tabla de auditoría.

Modelo de Base de Datos (Componente Obligatorio): antes de escribir código debe diseñar y sustentar el modelo de datos. La entrega incluye el modelo entidad-relación (MER), el modelo relacional normalizado hasta tercera forma normal (3FN), el diccionario de datos, el script DDL con llaves primarias, foráneas y restricciones, y un script DML con datos de prueba (mínimo diez registros por tabla principal).

El esquema debe evidenciar, como mínimo, los tres tipos de relación que se describen a continuación. En la sustentación deberá señalar en qué tablas se materializa cada una y por qué se modeló de esa forma:

- **Uno a uno (1:1).** La tabla usuario guarda únicamente las credenciales y el estado de la cuenta, mientras que perfil almacena los datos personales (nombres, apellidos, documento, teléfono, dirección y foto). La relación se garantiza declarando la llave foránea perfil.id_usuario como UNIQUE, de modo que un usuario no pueda tener dos perfiles. Se aceptan alternativas equivalentes, por ejemplo una relación 1:1 entre propiedad y ficha_tecnica.
- **Uno a muchos (1:N).** Una inmobiliaria publica muchas propiedades; una propiedad tiene muchas imágenes; un cliente genera muchas citas. La llave foránea se ubica siempre en el lado "muchos" y debe definirse con sus acciones referenciales (ON DELETE y ON UPDATE) debidamente justificadas.
- **Muchos a muchos (N:M).** Un usuario puede tener varios roles y un rol pertenece a varios usuarios, lo que se resuelve con la tabla intermedia usuario_rol. De igual forma, una propiedad posee varias características (piscina, parqueadero, ascensor, gimnasio) y cada característica aplica a varias propiedades, mediante la tabla propiedad_caracteristica. Las tablas intermedias llevan llave primaria compuesta y pueden incluir atributos propios, como la fecha de asignación o la cantidad.

Restricción UNIQUE: el modelo debe incluir al menos tres campos con la restricción UNIQUE. La aplicación está obligada a capturar el error que se produce al intentar duplicarlos y a mostrar un mensaje claro al usuario (por ejemplo, "el correo ya se encuentra registrado") en lugar de presentar una excepción de Java:

- usuario.correo (obligatorio): no pueden existir dos cuentas registradas con el mismo correo electrónico, pues es la credencial de ingreso.
- propiedad.matricula_inmobiliaria: identifica de manera irrepitable cada inmueble y evita publicaciones duplicadas.
- perfil.id_usuario y la llave compuesta usuario_rol(id_usuario, id_rol), que sostienen la relación 1:1 y evitan roles repetidos en la relación N:M.
- **Sugerida:** UNIQUE (id_propiedad, fecha_hora) en la tabla cita, para impedir que se agenden dos visitas a la misma propiedad en el mismo horario.

Entidades mínimas sugeridas: rol, usuario, usuario_rol, perfil, inmobiliaria, ciudad, tipo_propiedad, propiedad, imagen_propiedad, caracteristica, propiedad_caracteristica, cita, solicitud, documento_solicitud, favorito y auditoria. Puede agregar, renombrar o fusionar entidades siempre que justifique la decisión y se conserven los tres tipos de relación exigidos.

Consultas obligatorias: implemente y documente al menos cinco consultas que demuestren el dominio del modelo:

- Dos consultas con **INNER JOIN** entre tres o más tablas (por ejemplo, propiedades con su ciudad, su tipo y su inmobiliaria).
- Una consulta que resuelva una relación muchos a muchos (características de una propiedad o roles de un usuario).
- Una consulta con **LEFT JOIN** (por ejemplo, propiedades que aún no tienen citas agendadas).
- Una consulta de agregación con **GROUP BY** y **HAVING** que alimente uno de los reportes del sistema.

Requisitos Técnicos:

- **Tecnologías obligatorias:** Java EE (JSP), JDBC para el acceso a datos, HTML5, CSS3, JavaScript y Bootstrap para el frontend, y Apache Tomcat como servidor de aplicaciones.
- **Motores de base de datos:** una instancia local (MySQL, PostgreSQL u otra de su elección) y una instancia en línea y una local (MySQL o PostgreSQL). La cadena de conexión debe estar centralizada y ser configurable; no debe repetirse en cada clase.
- **Arquitectura:** separación en capas siguiendo el patrón MVC, con un controlador por entidad y JSPF.
- **Diseño responsivo:** la aplicación debe verse y operar correctamente en computador, tableta y celular, usando Bootstrap o media queries de CSS.
- **Validaciones e integridad:** campos obligatorios, formatos válidos (correo, teléfono, precio, fechas), manejo de excepciones SQL y mensajes de error comprensibles para el usuario final.
- **Documentación:** Diagramas (MER, modelo relacional y casos de uso) y la documentación Scrum del proyecto.

Estructura en Formato Scrum: el proyecto se divide en tres sprints de siete días cada uno. Simulará los roles de Scrum Master, Product Owner el Profesor y Development Team. Al inicio de cada sprint realizará un **Sprint Planning** (documento en un archivo .md las historias de usuario que abordará y su estimación); al cierre, un **Sprint Review** con demostración funcional y una **Sprint Retrospective** con las mejoras identificadas. Utilice un tablero de seguimiento (Trello, Jira o GitHub Projects) y Git con commits frecuentes y mensajes descriptivos. Se sugiere la siguiente distribución:

- **Sprint 1 – Cimientos y acceso:** MER, modelo relacional, y datos de prueba; conexión JDBC; landing page; registro e inicio de sesión con contraseñas y control de acceso por rol.
- **Sprint 2 – Núcleo del negocio:** CRUD de propiedades con imágenes (1:N) y características (N:M), buscador con filtros, perfil del usuario (1:1) y paneles diferenciados por rol.
- **Sprint 3 – Operación y cierre:** citas, solicitudes y documentos, favoritos, reportes con consultas de agregación, pruebas unitarias, despliegue y documentación final.

Product Backlog Inicial (Historias de Usuario Priorizadas): como Usuario, priorice estas historias que entrega el Docente Product Owner y agregue las que considere necesarias. Cada historia debe tener sus criterios de aceptación (DoD: Definition of Done).

1. Como visitante, quiero una página de aterrizaje atractiva para conocer la inmobiliaria y buscar propiedades rápidamente. (Prioridad Alta)
2. Como usuario, quiero registrarme con un correo único y validado para crear mi cuenta sin duplicados en el sistema. (Prioridad Alta)
3. Como usuario registrado, quiero iniciar y cerrar sesión de forma segura para que el sistema me lleve al panel que corresponde a mi rol. (Prioridad Alta)
4. Como administrador, quiero asignar y revocar roles a los usuarios para controlar los permisos de la aplicación. (Prioridad Alta)
5. Como cliente, quiero completar mi perfil con documento, teléfono y dirección asociados a mi cuenta para agilizar mis trámites. (Prioridad Media)
6. Como agente de la inmobiliaria, quiero registrar y editar propiedades con fotos, características y precio para mantener el catálogo actualizado. (Prioridad Alta)
7. Como cliente, quiero buscar y filtrar propiedades por ciudad, tipo, precio y características para encontrar las opciones que se ajusten a mis necesidades. (Prioridad Alta)

8. Como cliente, quiero marcar propiedades como favoritas para consultarlas más adelante sin tener que buscarlas de nuevo. (Prioridad Media)
9. Como cliente, quiero solicitar una cita en un horario disponible para visitar el inmueble sin que se crucen las agendas. (Prioridad Media)
10. Como cliente, quiero radicar los documentos de compra o arriendo y consultar el estado de mi solicitud. (Prioridad Media)
11. Como agente de la inmobiliaria, quiero aprobar o rechazar las solicitudes y sus documentos para dar trámite a la negociación. (Prioridad Media)
12. Como administrador, quiero un reporte de propiedades por ciudad y estado, generado con consultas de agregación, para tomar decisiones. (Prioridad Media)
13. Como administrador, quiero consultar la auditoría de accesos y cambios para hacer seguimiento a la operación del sistema. (Prioridad Baja)
14. Otras historias que usted proponga (integración de mapas, notificaciones por correo, comparador de propiedades, chat de contacto).

Entregables:

- Repositorio Git público con el código fuente y el como le fue en los tres sprints.
- Scripts .sql y los modelos MER y relacional exportados en imagen o PDF.
- Documentación Scrum: Evidencia del tablero padlet de seguimiento de su proyecto.
- Aplicación funcionando en ambiente local y en línea de la base de datos y de la aplicación otorga puntos adicionales.
- Sustentación en grupo de a dos o individual, en la que se expondrá el proyecto.

Criterios de Evaluación:

CRITERIO	EVIDENCIA ESPERADA	PUNTAJE
Aplicación web funcional y control de acceso por rol	Login con contraseña cifrada, filtro de rutas por rol, paneles diferenciados, CRUD de propiedades, citas y solicitudes en operación.	40
Aplicación de la metodología Scrum	Tres sprints documentados, tablero de seguimiento y repositorio Git con historial de commits.	30
Modelo de datos y documentación	MER y modelo relacional en 3FN, relaciones 1:1, 1:N y N:M identificadas, restricciones UNIQUE, script DDL/DML, consultas con JOIN y sustentación individual.	30
TOTAL		100

Nota: Un proyecto que funcione pero cuyo autor no sepa explicar su modelo de datos ni su control no obtiene el puntaje completo. La sustentación es en grupo presencial los dos deben dominar el tema y obligatoria.